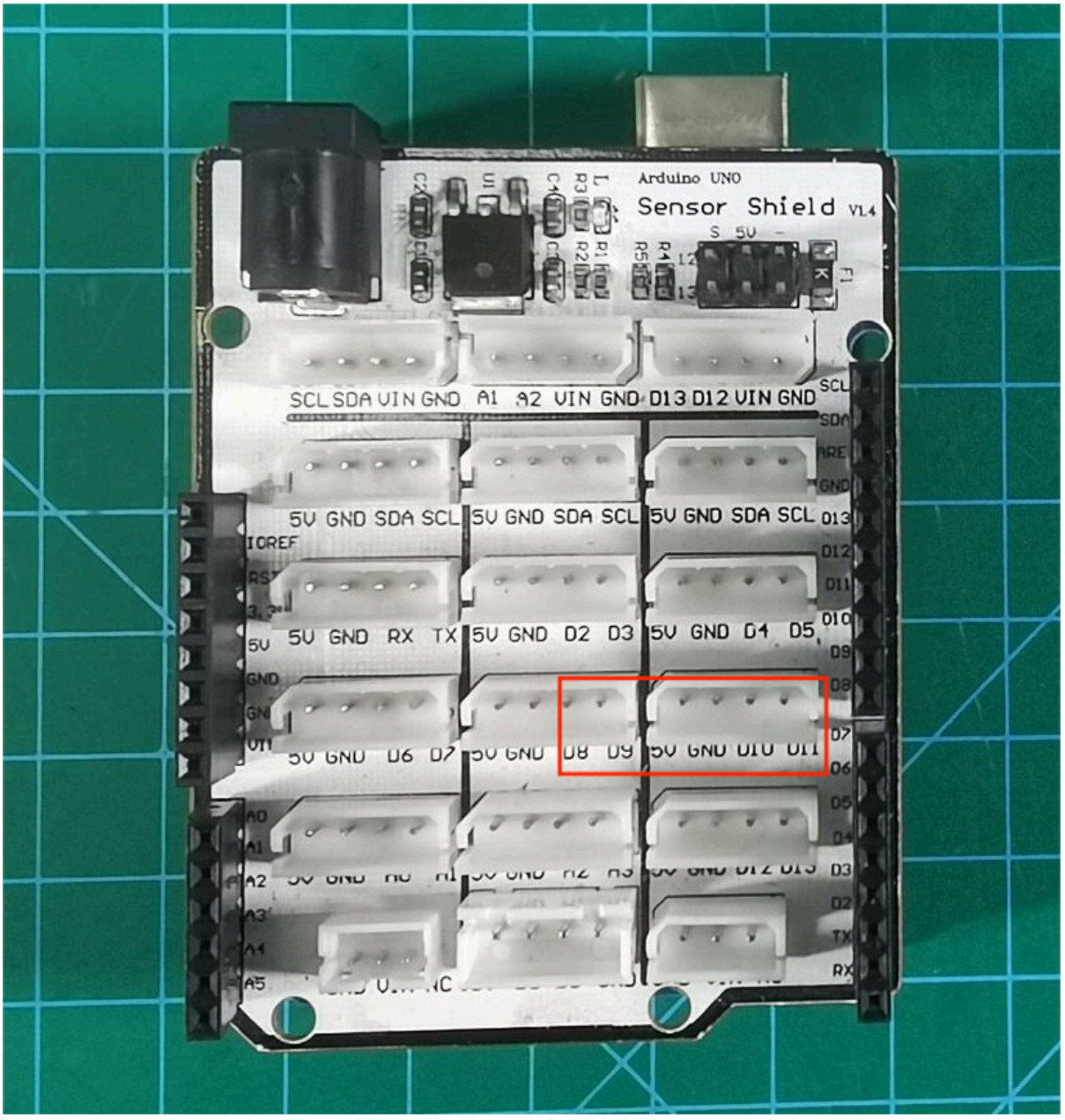
3. 示例程序

3.1 四路巡线Arduino例程

3.1.1 准备工作：

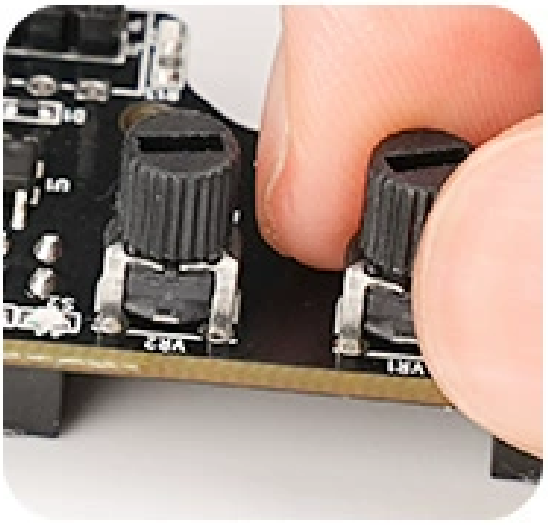
本节示例采用Arduino UNO+Arduino扩展板，传感器需要使用6P线连接至Arduino扩展板上的5V、GND和4个IO口。



引脚	引脚说明
5V	电源输入
GND	电源地
S1	D4
S2	D5

引脚	引脚说明
S3	D3
S4	D2

> **Note**
>
> **注意：可以通过旋钮调节距离，通过调节探测距离的远近，来调整不同环境的调节灵敏度， 调节步骤是先将红外探头放置在黑色区域，然后将所有的蓝色灯调亮，再慢慢转动旋钮调节至刚好熄灭，最后探头接触到白色区域后，刚好亮起，至此调节到了探头的最佳灵敏度。如图所示：**



3.1.2 示例程序：

此巡线传感器是红外探测法为工作原理的传感器。这个传感器上有 4 个探头，每个探头有一个接收管和一个发射管。

1. 将装有扩展板的 Arduino UNO开发板通过 USB数据线连入电脑。
2. 打开“示例程序”内的 Arduino示例程序。
3. 选择正确的开发板及端口（本节以 UNO为例，示例所用端口号为11，用户需根据实际情况自行选择）



3.1.3 实现效果：

1. （该资料适合用于麦轮，差速，履带，阿克曼小车）
2. 取一张带黑线的白纸（或者在白纸上画一些黑线）
3. 将传感器探头对准黑白线，同时可打开 Arduino IDE 的串口监视器，四个探头会将探测的数据以“1 或者 0”的形式打印出来。黑线对应灯光熄灭时显示数值为 0，黑线对应灯光亮起时显示数值为 1。



3.1.4 程序分析：

使用4个巡线传感器检测黑线，并将检测到的传感器电平值通过串口打印出来，巡线传感器在检测到黑线时输出的电平值为0，检测到白线时输出的电平值为1，程序如下图所示：

```
#include <Arduino.h>

const int SENSOR1_PIN = 8;
const int SENSOR2_PIN = 9;
const int SENSOR3_PIN = 10;
const int SENSOR4_PIN = 11;


void setup() {
  // put your setup code here, to run once:
  pinMode(SENSOR1_PIN,INPUT);
  pinMode(SENSOR2_PIN,INPUT);
  pinMode(SENSOR3_PIN,INPUT);
  pinMode(SENSOR4_PIN,INPUT);

  Serial.begin(9600);
}

void loop() {
  // put your main code here, to run repeatedly:
```

```
  int S1 = digitalRead(SENSOR1_PIN);
  int S2 = digitalRead(SENSOR2_PIN);
  int S3 = digitalRead(SENSOR3_PIN);
  int S4 = digitalRead(SENSOR4_PIN);

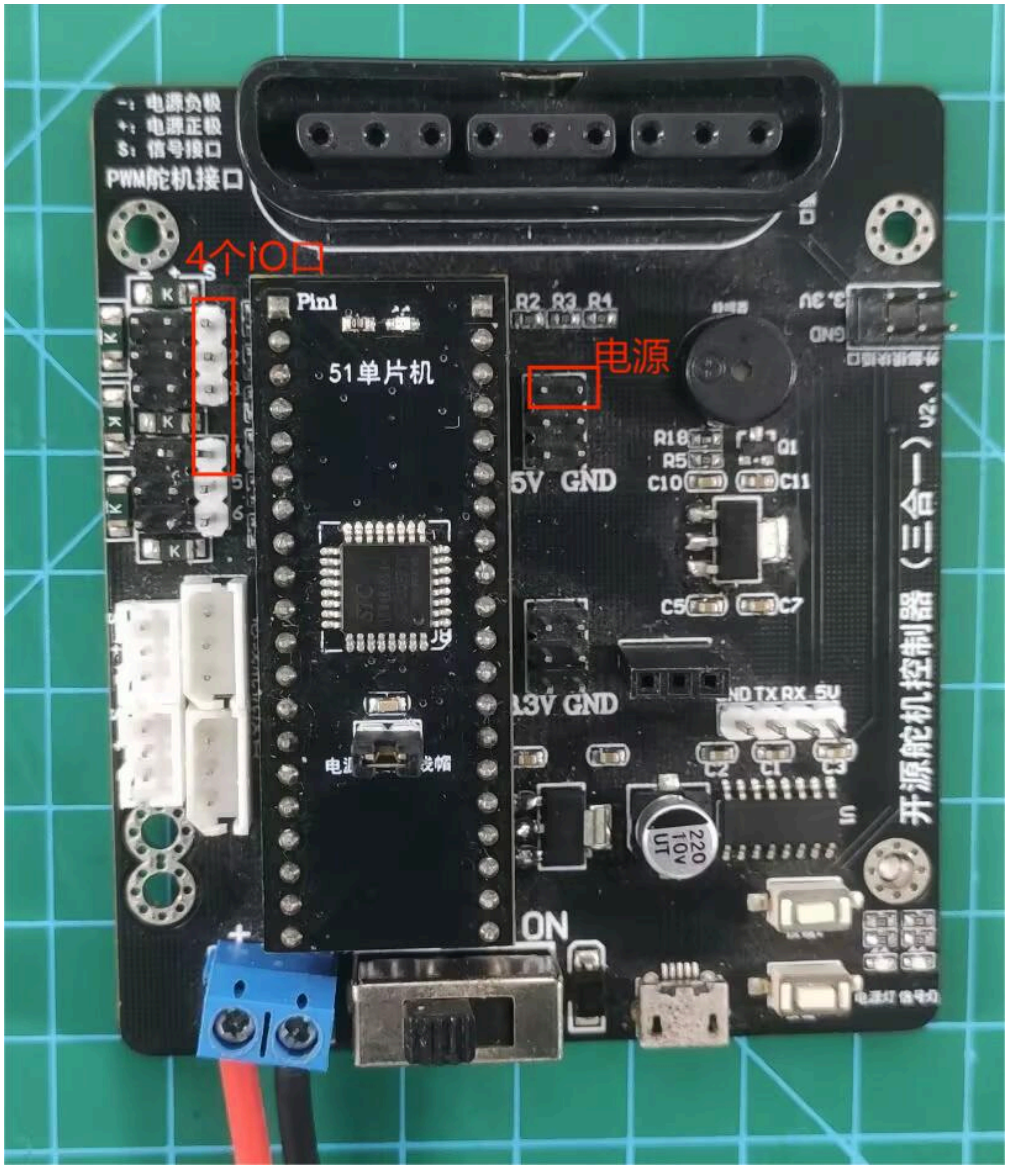
  Serial.print("S1: ");
  Serial.print(S1);
  Serial.print(" | S2: ");
  Serial.print(S2);
  Serial.print(" | S3: ");
  Serial.print(S3);
  Serial.print(" | S4: ");
  Serial.println(S4);

  delay(500);
}
```

3.2 四路巡线C51例程

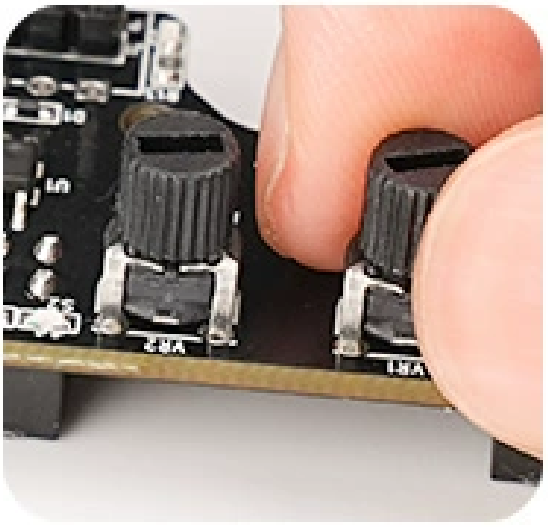
3.2.1 准备工作：

本节示例采用STC15W4K16S4主控芯片+开源舵机控制器扩展板，传感器需要使用6P线连接至扩展板上的5V、GND和4个IO口。



引脚	引脚说明
5V	电源输入
GND	电源地
S1	P24
S2	P54
S3	P55
S4	P32

> **Note**
>
> **注意：可以通过旋钮调节距离，通过调节探测距离的远近，来调整不同环境的调节灵敏度， 调节步骤是先将红外探头放置在黑色区域，然后将所有的蓝色灯调亮，再慢慢转动旋钮调节至刚好熄灭，最后探头接触到白色区域后，刚好亮起，至此调节到了探头的最佳灵敏度。如图所示：**



3.2.2 示例程序：

此巡线传感器是红外探测法为工作原理的传感器。这个传感器上有 4 个探头，每个探头有一个接收管和一个发射管。

1. 将装有扩展板的开发板通过 USB数据线连入电脑，或者使用拓展串口。
2. 打开“示例程序”内的C51示例程序。参考“**2. 软件安装\51单片机\开发环境搭建**”，进行程序烧录，注意烧录时要选对 输入用户程序运行时的IRC频率 12.000 MHz ，根据实际检测得到的频率选择相对应的数值。

3.2.3 实现效果：

1. （该资料适合用于麦轮，差速，履带，阿克曼小车）
2. 取一张带黑线的白纸（或者在白纸上画一些黑线）
3. 将传感器探头对准黑白线，同时可打开串口监视器，四个探头会将探测的数据以“1 或者 0”的形式打印出来。黑线对应灯光熄灭时显示数值为 1，黑线对应灯光亮起时显示数值为 0。（要注意检查串口参数是否设置正确。）



3.2.4 程序分析：

使用4个巡线传感器检测黑线，并将检测到的传感器电平值通过串口打印出来，巡线传感器在检测到黑线时输出的电平值为1，检测到白线时输出的电平值为0，程序如下图所示：

```
#include <STC15.h>  //兼容STC11xx系列

typedef unsigned short  u16, U16, uint16, UINT16, WORD;
```

```c
// 延迟函数
void DelayMs(uint16 ms)
{
  uint16 i,j;
  for(i=0;i<800;i++) // 外循环
  for(j=0;j<ms;j++);  // 内循环用于实现指定的延迟
}

// 初始化UART1的函数
void InitUart1(void)
{
  SCON = 0x50;     // 设置串行控制寄存器，工作方式1,8位可变波特率
  AUXR |= 0x40;    // 串口1波特率独立倍增位，时钟1T模式
  AUXR &= 0xFE;    // 禁用定时器1波特率加倍
  TMOD &= 0x0F;    // 定时器1为方式1，16位自动重装
  TL1 = 0xC7;    // 设置定时器初值
  TH1 = 0xFE;    // 设置定时器初值
  ET1 = 0;     // 禁用定时器1中断
  TR1 = 1;     // 开启定时器1
  ES = 1;      // 使能串行口中断
  TI = 1;          // 清零发送标志
}

// 串口发送数据函数
void Uart1SendData(unsigned char dat)
{
  SBUF = dat;                       // 将数据放入串行缓冲寄存器
  while (!TI);  // 等待数据发送完成
  TI = 0;       // 清零发送完成标志
}

// 打印GPIO电平状态函数
void printGPIOLevel(void)
{
    // 检测GPIO口电平，如果高电平，发送'1'；否则，发送'0'
    Uart1SendData(P24 ? '1' : '0');
    Uart1SendData(P54 ? '1' : '0');
    Uart1SendData(P55 ? '1' : '0');
    Uart1SendData(P32 ? '1' : '0');

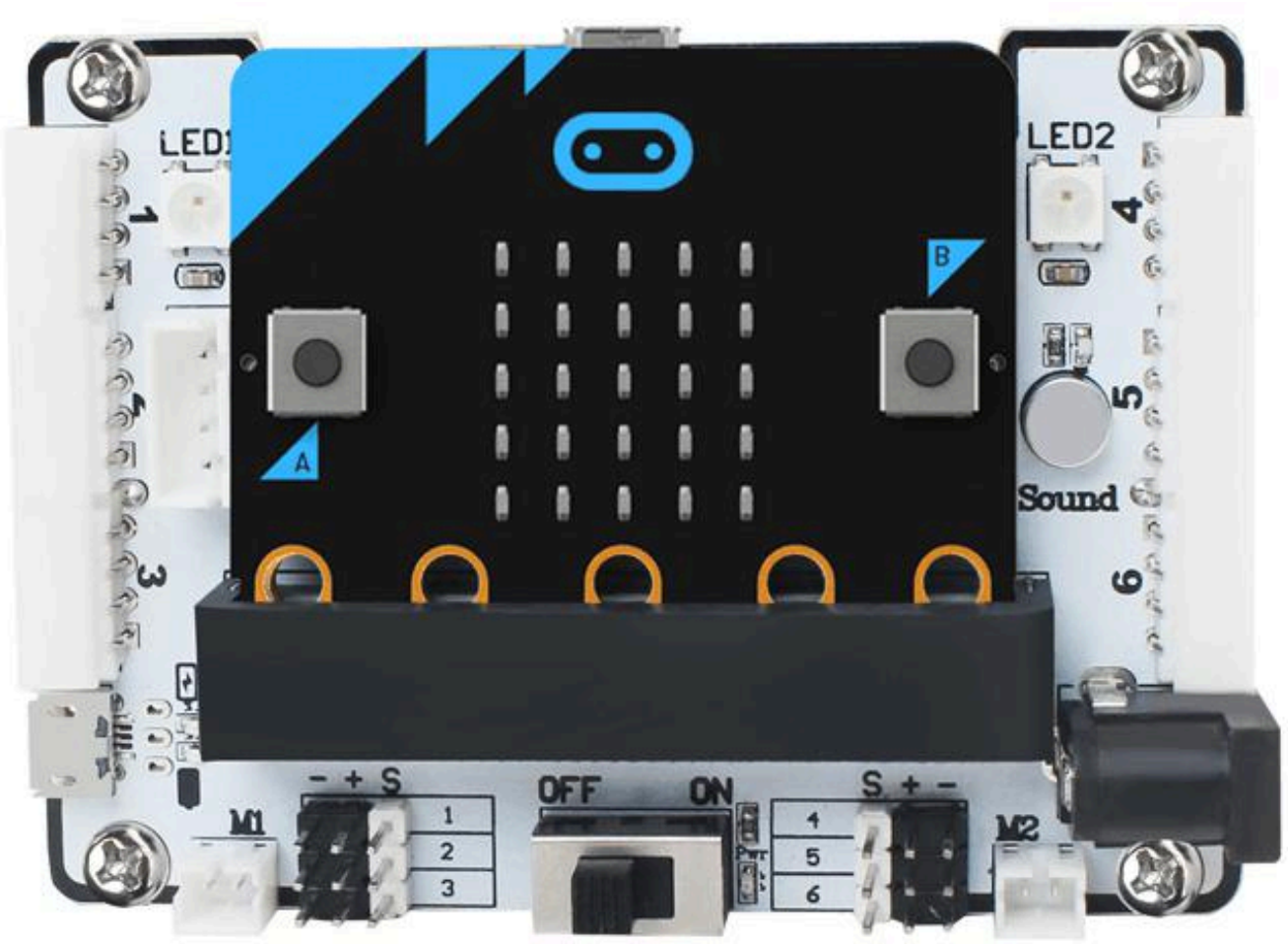
    Uart1SendData('\n');  // 发送换行符
}

// 主函数
int main(void)
{
    Uart1Init();  // 初始化UART1
    while(1)
    {
        printGPIOLevel(); // 打印GPIO电平状态
        DelayMs(500);  // 延迟500ms
    }
    return 0;
}
```

3.3 四路巡线micro:bit例程

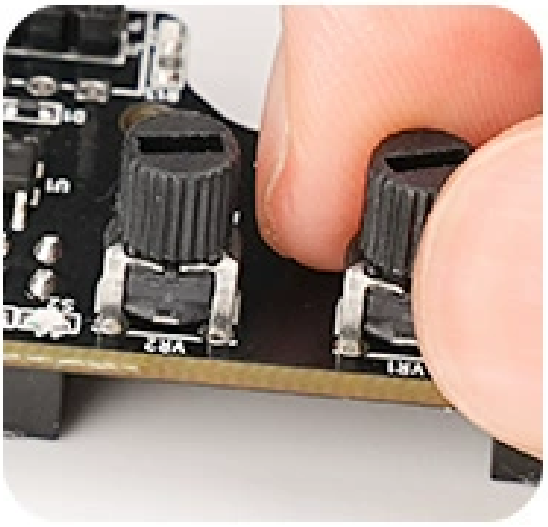
3.3.1 准备工作:

本节示例使用的是 micro:bit 主板和 micro:bit 扩展板。需要将传感器通过 4P 线连接至micro:bit 扩展板的 4 号接口。



引脚	引脚说明
5V	电源输入
GND	电源地
S1	P1
S2	P2
S3	P13
S4	P14

> **Note**
>
> **注意：可以通过旋钮调节距离，通过调节探测距离的远近，来调整不同环境的调节灵敏度， 调节步骤是先将红外探头放置在黑色区域，然后将所有的蓝色灯调亮，再慢慢转动旋钮调节至刚好熄灭，最后探头接触到白色区域后，刚好亮起，至此调节到了探头的最佳灵敏度。如图所示：**



3.3.2 示例程序：

此巡线传感器是红外探测法为工作原理的传感器。这个传感器上有 4 个探头，每个探头有一个接收管和一个发射管。

1. 点击 https://makecode.microbit.org/，进入 micro:bit 官网在线编程平台。
2. 可点击网页中的对应按钮（如下图所示），或将本示例的 micro:bit 程序直接拖动到网页页面内。



3. 将 micro:bit 主板插入电脑任意一个 USB 接口，把程序下载至该 micro:bit 所在的盘 符内。
4. 下载完成后，将 micro:bit 主板插入 micro:bit 扩展板内即可。

3.3.3 实现效果：

1. 取一张带黑线的白纸（或者白纸上画一些黑线）。
2. 将传感器探头对准黑白线，micro:bit 主板将会打印“01234”的字样出来。

3.4 四路巡线传感器STM32例程

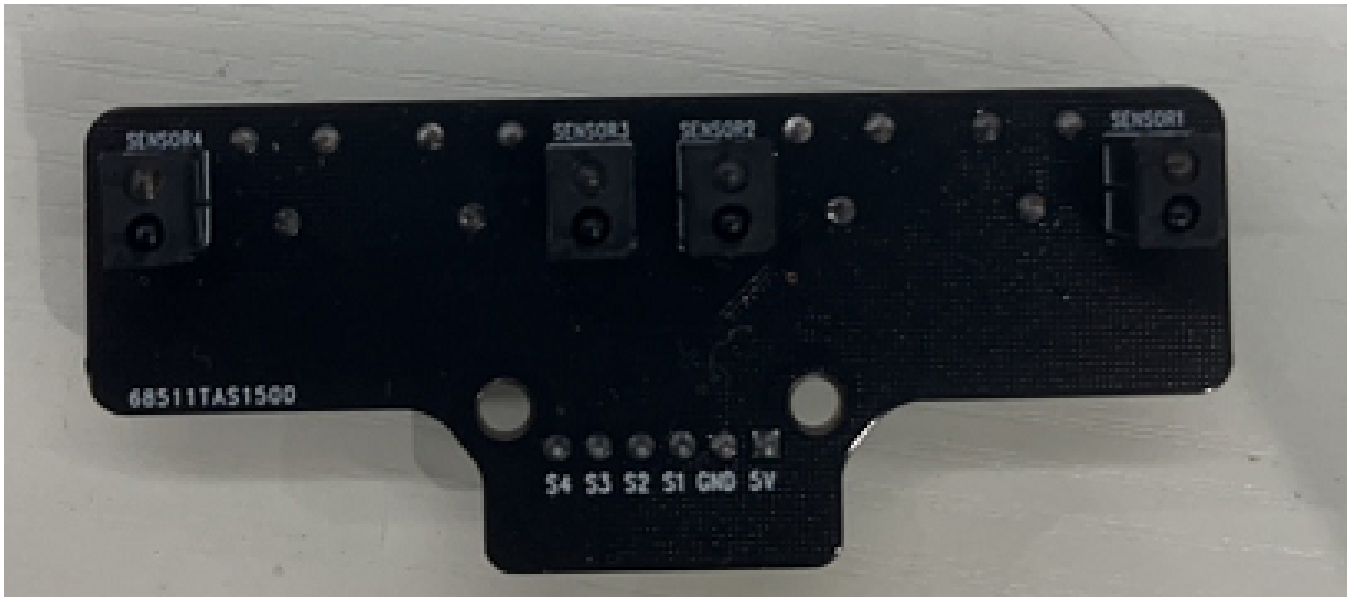
3.4.1 简介

此巡线传感器是红外探测法为工作原理的传感器。这个传感器上有 4 个探头，每个探头有一个接收管和一个发射管。

当红外反射越强(白色)时，输出越大，红外放射越弱（黑色），输出越小。探测器离黑色线越近输出越小，由此可以通过输出模拟量判断黑线的距离远近，数值越小的传感器离黑线越近。传感器的灵敏度可以通过调节传感器上的旋钮来调整阈值，顺时针转动可以增强灵敏度，逆时针则减弱灵敏度。

因此这款传感器可以识别黑白线，应用于智能小车、机器人的巡线运动，且可巡较为复杂的路线。同时，这款传感器的四路巡线探头合理优化间距布局，内侧两路在黑线内精准识别、外侧两路可提前探测到大弯道的黑线并提前预警；具有探测速度快、适应性良好等优点。下图为传感器示意图。



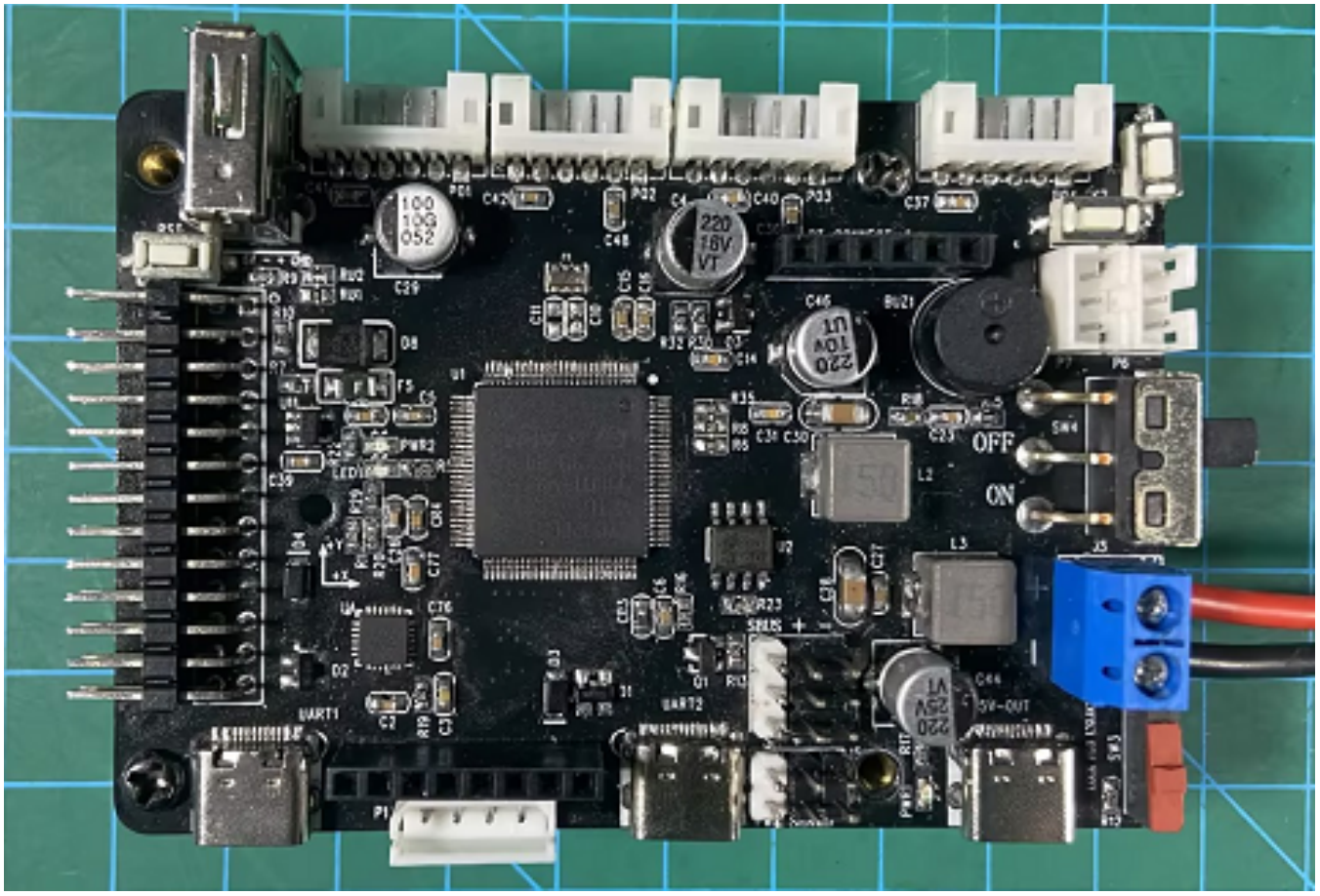


3.4.2 程序路径

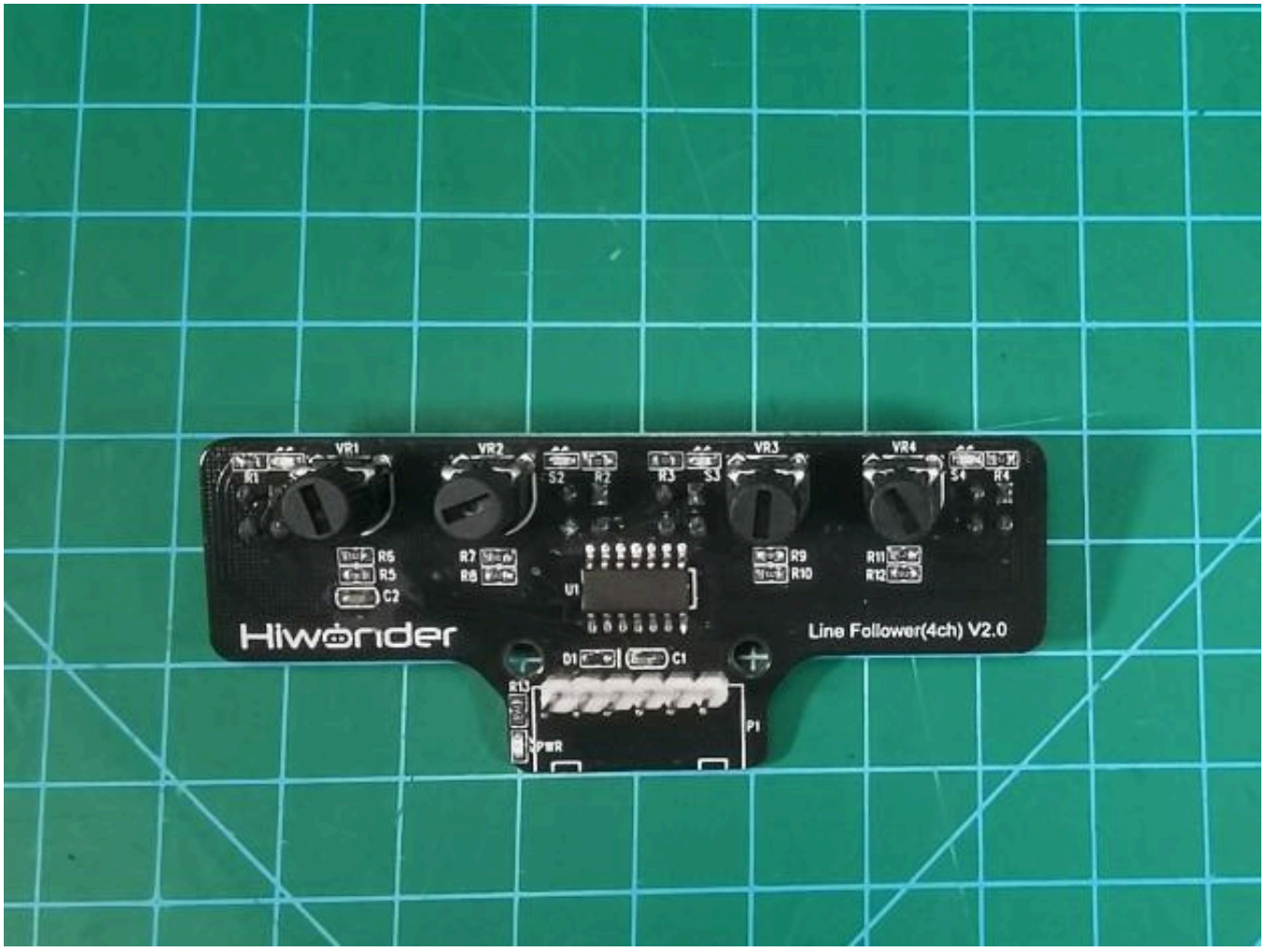
本小节内容实现的源码位于“**四路巡线\程序附录\RosRobotControllerM4-line_follower**”目录下的`Hiwonder/System/line_follower.c`，可使用 Keil5 打开工程文件（具体安装方式可参考前文）。

3.4.3 硬件连接说明

1. STM32 主控板；


OFF
ON
UART1
UART2
5V-OUT

2. 四路巡线传感器；


VR1
VR2
VR3
VR4
Hiwonder
Line Follower(4ch) V2.0
PWR

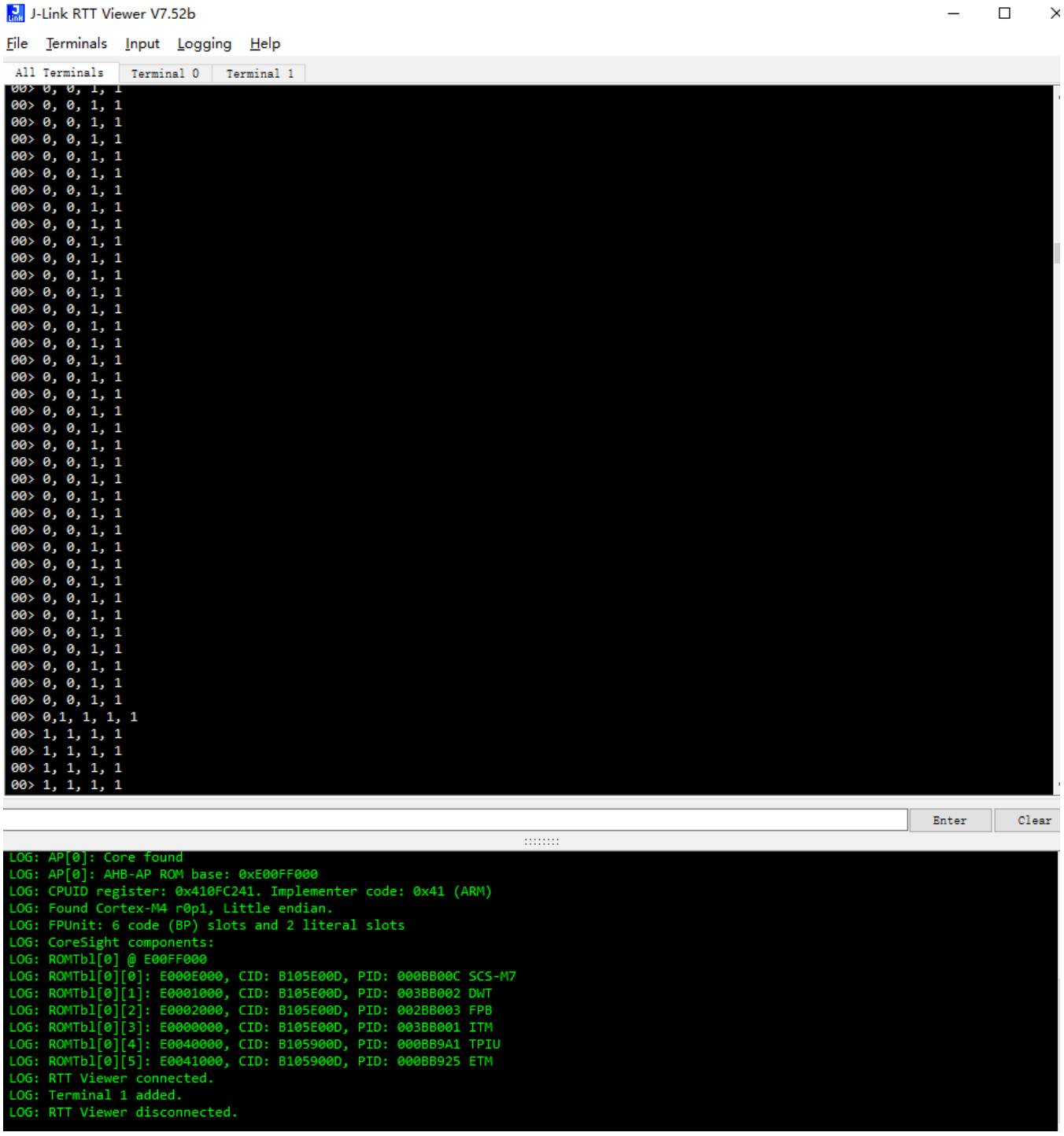


3.4.4 接口说明：

引脚	引脚说明
5V	电源输入
GND	电源地
S1	PC1
S2	PC0
S3	PE3
S4	PE2

3.4.5 实例结果：

使用“J-Link RTT Viewer”软件进行打印，如下图所示：



打印的内容0、1为四路巡线的接收到黑线，白线。

3.4.6 参数说明：

规格参数

（该资料适合用于麦轮，差速，履带，阿克曼小车）

工作电压	3.3V-5V
工作电流	10mA-50mA
工作温度	0°C~50°C

灵敏度调节	微型电位器调节
通讯方式	GPIO输入
PWR 说明	传感器供电后亮起
具有 4 个探头，每个探头都有一个红外发射器和一个红外接收器。 4 个探头分别独立工作。	

源码分析

在这里我们运用了STM32F4 HAL库设置了STM32F407VET6的PC0、PC1、PE2、PE3为输入检测，如下图所示：

```
void Error_Handler(void);

/* USER CODE BEGIN EFP */

/* USER CODE END EFP */

/* Private defines -----------------------------------------------------------*/
#define LF_S4_Pin GPIO_PIN_2
#define LF_S4_GPIO_Port GPIOE
#define LF_S3_Pin GPIO_PIN_3
#define LF_S3_GPIO_Port GPIOE
#define LF_S2_Pin GPIO_PIN_0
#define LF_S2_GPIO_Port GPIOC
```

通过PID控制：

```
float line_folower(float kp, float ki, float kd)
{
    static float error = 0, last_error = 0, integral = 0;
    bool s1 = HAL_GPIO_ReadPin(LF_S1_GPIO_Port, LF_S1_Pin);
    bool s2 = HAL_GPIO_ReadPin(LF_S2_GPIO_Port, LF_S2_Pin);
    bool s3 = HAL_GPIO_ReadPin(LF_S3_GPIO_Port, LF_S3_Pin);
    bool s4 = HAL_GPIO_ReadPin(LF_S4_GPIO_Port, LF_S4_Pin);
```

这个程序是读取每个巡线传感器的状态。将每个传感器状态存储在一个布尔变量中（s1，s2，s3，s4）

函数`line_follower`**三个参数**，分别代表PID控制器的比例、积分和微分系数。

定义三个静态浮点变量：

1. `error`（当前的误差）；
2. `last_error`（上一次的误差）；
3. `integral`（误差积分）。

```
    if( !s1 && s2 && !s3 && !s4) { /* 0 1 0 0 */
        error = -1;
    } else if(s1 && s2 && !s3 && !s4) { /* 1 1 0 0 */
        error = -2;
    } else if(s1 && !s2 && !s3 && !s4) { /* 1 0 0 0 */
```

```
        error = -6;
    } else if(!s1 && !s2 && s3 && !s4) { /* 0 0 1 0 */
        error = 1;
    } else if(!s1 && !s2 && s3 && s4) { /* 0 0 1 1 */
        error = 2;
    } else if(!s1 && !s2 && !s3 && s4) { /* 0 0 0 1 */
        error = 6;
    } else {
        error = 0;
```

这个条件（if-else）结构，用于根据四路巡线传感器的状态计算错误值。

每种情况都对应于一个特定的传感器组合，给出不同的错误值。（**详情可以看最下面有详细分析**）

```
    integral += error;
    integral = integral > 80 ? 80 : (integral < -80 ? -80 : integral);
    float output = kp * error + ki * integral + kd * (error - last_error);
    return output;
```

这个程序通过返回PID控制器的输出值，根据这个值用于调整底盘的角速度以跟踪线路。

这个是计算错误的积分（integral += error;）。然后，对积分进行限制，确保其值在-80到80之间。

```
    integral = integral > 80 ? 80 : (integral < -80 ? -80 : integral);
```

这个部分程序对积分值进行了限制，防止积分过大，将积分值被限制在-80到80之间。

```
    float output = kp * error + ki * integral + kd * (error - last_error);
    return output;
```

这个程序计算了PID控制的输出，kp、ki、kd是比例、积分、微分的系数。然后返回PID控制器的输出，根据输出可以调整小车底盘的行驶方向。

kp * error计算的是比例项，它使得控制器的输出和当前的位置偏移成正比；

ki * integral计算的是积分项，它使得控制器的输出和过去所有的位置偏移的累积成正比；

kd * (error - last_error)计算的是微分项，它使控制器的输出和位置偏移的变化速度成正比。

```
    if( !s1 && s2 && !s3 && !s4) { /* 0 1 0 0 */
        error = -1;
    } else if(s1 && s2 && !s3 && !s4) { /* 1 1 0 0 */
        error = -2;
    } else if(s1 && !s2 && !s3 && !s4) { /* 1 0 0 0 */
        error = -6;
    } else if(!s1 && !s2 && s3 && !s4) { /* 0 0 1 0 */
        error = 1;
    } else if(!s1 && !s2 && s3 && s4) { /* 0 0 1 1 */
        error = 2;
    } else if(!s1 && !s2 && !s3 && s4) { /* 0 0 0 1 */
        error = 6;
    } else {
```

```
        error = 0;
    }
```

接上面程序的详细拆分这部分代码是根据四路巡线传感器的读数（s1, s2, s3, s4）来确定当前小车底盘的位置偏移，也即“错误”(error)。这个错误值将被用于PID控制，以调整小车的行驶方向。

传感器的读数是布尔值，true表示该传感器上检测到了线路，false则表示没有检测到，在这个程序中，我们四路巡线传感器s2和s3在机器人的正中，s1和s4分别在机器人的左侧和右侧。

下面是对每个条件的详细解析：

```
    if( !s1 && s2 && !s3 && !s4) { /* 0 1 0 0 */
        error = -1;
```

这表示当只有s2检测到线路时，小车底盘偏离线路的方向是向左，所以设定一个负的错误值-1。

```
    } else if(s1 && s2 && !s3 && !s4) { /* 1 1 0 0 */
        error = -2;
```

这表示当s1和s2检测到线路时，小车底盘更偏离线路的方向是向左，所以设定一个更大的负错误值-2。

```
    } else if(s1 && !s2 && !s3 && !s4) { /* 1 0 0 0 */
        error = -6;
```

这表示当只有s1检测到线路时，小车底盘严重地偏离线路的方向是向左，所以设定一个更大的负错误值-6。

```
    } else if(!s1 && !s2 && s3 && !s4) { /* 0 0 1 0 */
        error = 1;
```

这表示当只有s3检测到线路时，小车底盘偏离线路的方向是向右，所以设定一个错误值-1。

```
    } else if(!s1 && !s2 && s3 && s4) { /* 0 0 1 1 */
        error = 2;
```

这表示当s3和s4检测到线路时，小车底盘更偏离线路的方向是向右，所以设定一个更大的错误值+2。

```
    } else if(!s1 && !s2 && !s3 && s4) { /* 0 0 0 1 */
        error = 6;
```

这表示当只有s4检测到线路时，小车底盘严重地偏离线路的方向是向右，所以设定一个更大的错误值+6。

```
    } else {
        error = 0;
    }
```

这表示当其他情况，如所有传感器都没有检测到线路，或者所有传感器都检测到线路时，程序认为小车底盘没有偏离线路，所以设定错误值为0。

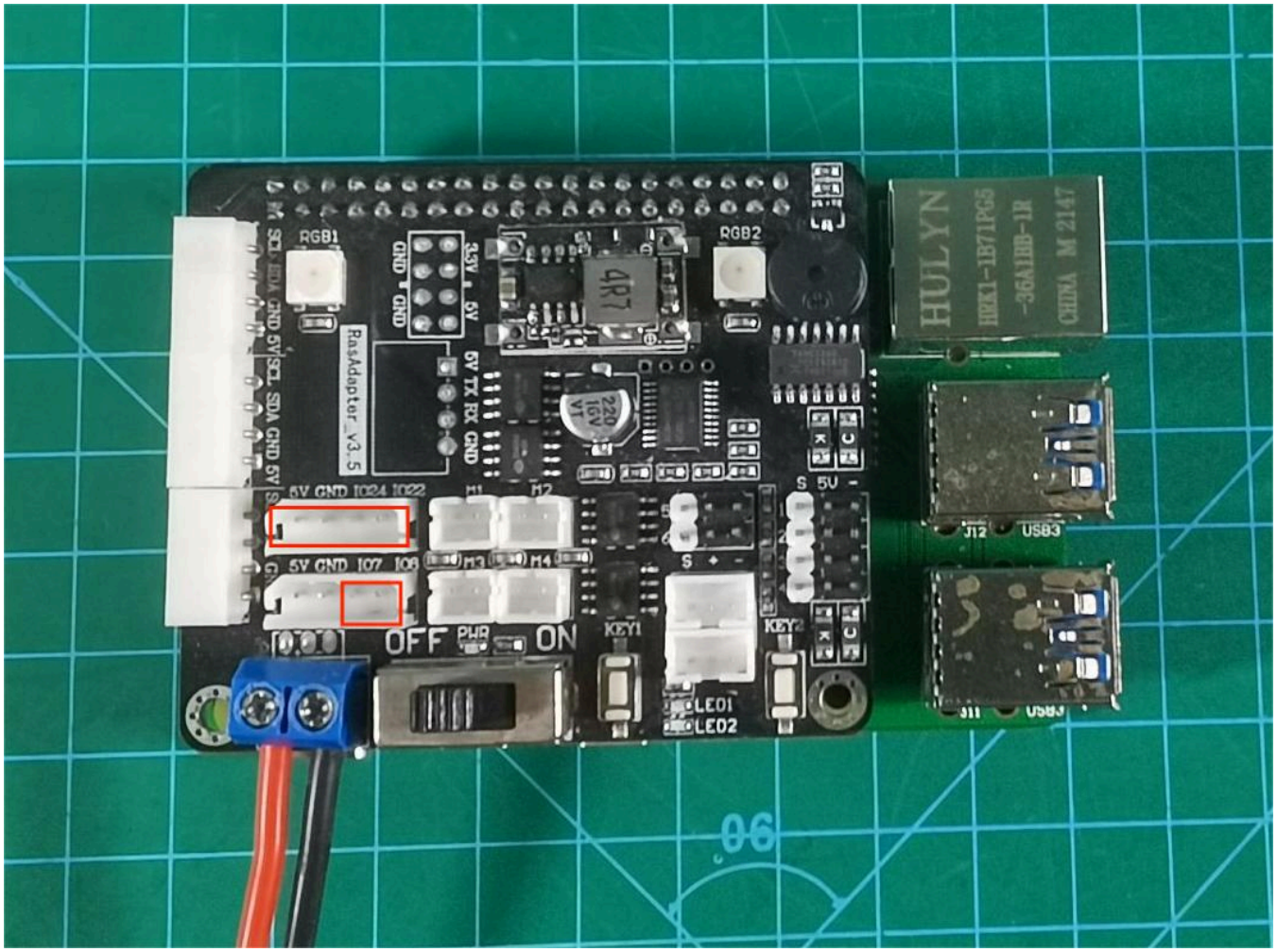
> **Note**

注意：【切勿正负极接反】

3.5 四路巡线树莓派例程

3.5.1 准备工作：

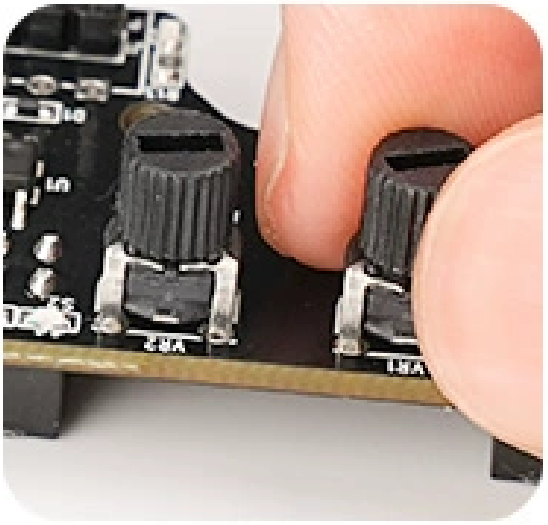
本节示例采用树莓派4B+扩展板，传感器需要使用6P线连接至树莓派扩展板上的5V、GND和4个IO口。



引脚	引脚说明
5V	电源输入
GND	电源地
S1	GPIO.8 对应 BOARD 编码：24
S2	GPIO.7 对应 BOARD 编码：26
S3	GPIO.24 对应 BOARD 编码：18
S4	GPIO.22 对应 BOARD 编码：15

Note

> **注意：可以通过旋钮调节距离，通过调节探测距离的远近，来调整不同环境的调节灵敏度， 调节步骤是先将红外探头放置在黑色区域，然后将所有的蓝色灯调亮，再慢慢转动旋钮调节至刚好熄灭，最后探头接触到白色区域后，刚好亮起，至此调节到了探头的最佳灵敏度。如图所示：**



3.5.2 示例程序：

此巡线传感器是红外探测法为工作原理的传感器。这个传感器上有 4 个探头，每个探头有一个接收管和一个发射管。

1. 电脑使用VNC Viewer 

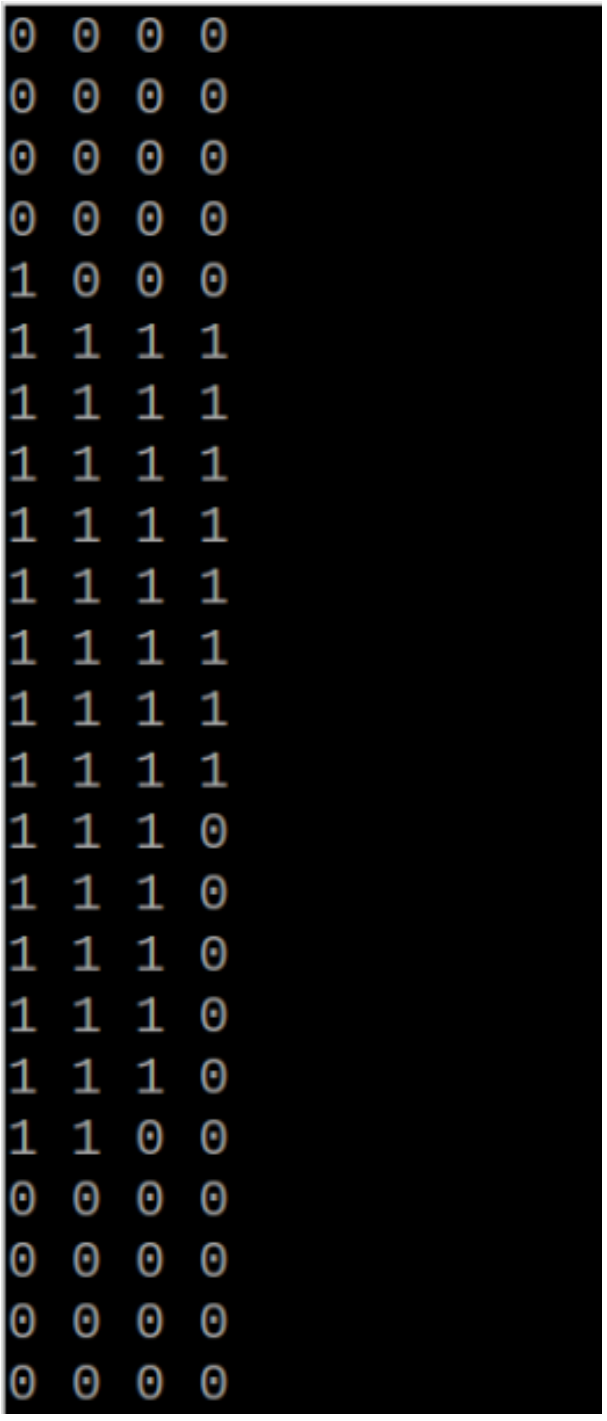
软件与树莓派连接。
2. 使用MobaXterm  软件将“Line_Follower.py”的树莓派的示例程序导入树莓派
3. 在Line_Follower.py例程存放的路径下使用如下命令执行程序文件：

```
sudo python3 Line_Follower.py
```

```
hiwonder@raspberrypi:~/Demo $ sudo python3 Line_Follower.py
```

3.5.3 实现效果：

1. （该资料适合用于麦轮，差速，履带，阿克曼小车）
2. 取一张带黑线的白纸（或者在白纸上画一些黑线）
3. 将传感器探头对准黑白线，同时可在终端窗口查看打印数据。四个探头会将探测的数据以“1 或者 0”的形式打印出来。黑线对应灯光熄灭时显示数值为 0，黑线对应灯光亮起时显示数值为 1。



3.5.4 程序分析：

使用4个巡线传感器检测黑线，并将检测到的传感器电平值通过串口打印出来，巡线传感器在检测到黑线时输出的电平值为0，检测到白线时输出的电平值为1，程序如下图所示：

```
import RPi.GPIO as GPIO
import time

# 设定GPIO引脚编号模式为BOARD模式
GPIO.setmode(GPIO.BOARD)

# 定义巡线传感器连接的引脚编号
SENSOR1_PIN = 18  # 这里的引脚编号需要根据实际情况进行修改
SENSOR2_PIN = 15
```

```
SENSOR3_PIN = 26
SENSOR4_PIN = 24


# 将巡线传感器引脚设置为输入模式
GPIO.setup(SENSOR1_PIN, GPIO.IN)
GPIO.setup(SENSOR2_PIN, GPIO.IN)
GPIO.setup(SENSOR3_PIN, GPIO.IN)
GPIO.setup(SENSOR4_PIN, GPIO.IN)

try:

    while True:

        # 读取巡线传感器的电平值，如果传感器上有黑线，返回1，否则返回0
        sensor1Value = GPIO.input(SENSOR1_PIN)
        sensor2Value = GPIO.input(SENSOR2_PIN)
        sensor3Value = GPIO.input(SENSOR3_PIN)
        sensor4Value = GPIO.input(SENSOR4_PIN)

        # 打印每个传感器读取的电平值
        print(sensor1Value, sensor2Value, sensor3Value, sensor4Value)
        # 等待0.1秒（100毫秒），降低打印频率
        time.sleep(0.1)

finally:
    GPIO.cleanup()  # 程序结束后，清理所有GPIO设置
```